

Database Design Document

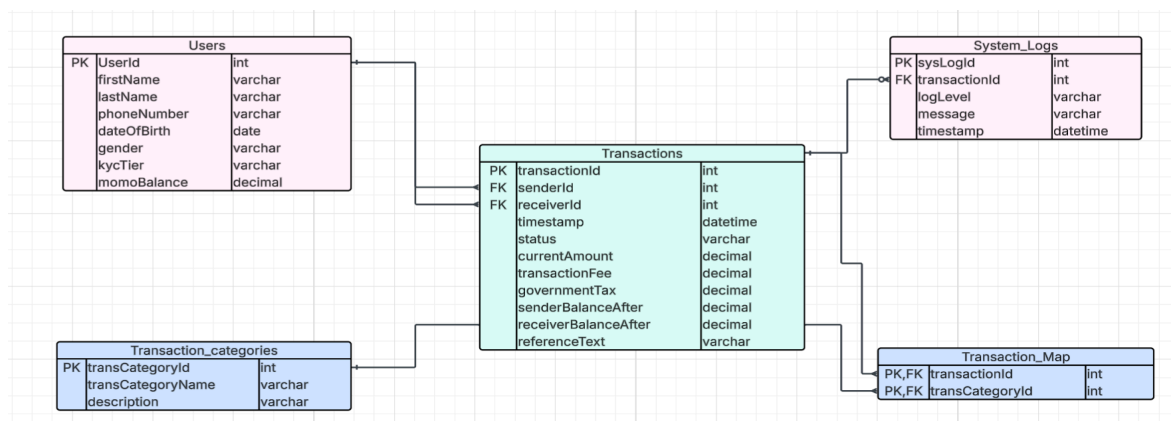
ERD Documentation

Our database design centers on five tables: Users, Transactions, Transaction_Categories, Transaction_Map (a junction table), and System_Logs. We chose a single, unified Users entity rather than separate Sender and Receiver tables because the same phone number can act as either role across different transactions; separating them would duplicate personal data and risk inconsistency if a user's details changed. Instead, the Transactions table references Users twice, through senderId and receiverId foreign keys, capturing both roles without redundancy. We excluded authentication data such as PINs or OTPs from our schema, since storing sensitive security credentials in a database, even for academic purposes, does not reflect responsible data-handling practice; real financial systems never store such data in plain, queryable form. For balances, we store only senderBalanceAfter and receiverBalanceAfter, since MoMo SMS messages report only the balance visible to the message's recipient after the transaction completes; a "before" balance is not realistically extractable from the source data.

The relationship between Transactions and Transaction_Categories is many-to-many, since a single transaction may reasonably be tagged with more than one category, for example, a Merchant Payment that is also a Bill Payment, while a category naturally applies to many transactions. This relationship is resolved through the Transaction_Map junction table, following standard relational design practice, since relational tables cannot directly store multiple values in a single field.

Finally, System_Logs exists independently of the financial data to track our own ETL pipeline's behavior. Its transactionId foreign key is nullable, since some log entries, such as parsing failures on malformed SMS text, occur before any transaction record can be created. One transaction may generate zero, one, or many log entries, but each log entry links to at most one transaction.

ERD Diagram



Data Dictionary:

Table	Column	Data Type	Key	Description
users	userId	INT	PK	Unique identifier for each user
users	firstName	VARCHAR(50)		User first name
users	lastName	VARCHAR(50)		User last name
users	phoneNumber	VARCHAR(15)	UNIQUE	Mobile money subscriber MSISDN
users	dateOfBirth	DATE		User date of birth
users	gender	VARCHAR(10)		User gender
users	kycTier	VARCHAR(20)		KYC verification level
users	momoBalance	DECIMAL(12,2)		Current mobile money balance (≥ 0)
users	identifier	VARCHAR(50)		National ID or external system identifier
transactions	transactionId	INT	PK	Unique identifier for each transaction
transactions	senderId	INT	FK -> users.userId	The user sending the money
transactions	receiverId	INT	FK -> users.userId	The user receiving the money
transactions	transactionTimestamp	DATETIME		Date and time the transaction occurred
transactions	status	VARCHAR(20)		Transaction status: success, failed, pending
transactions	currentAmount	DECIMAL(12,2)		Transaction amount (≥ 0)
transactions	transactionFee	DECIMAL(10,2)		Fee charged for the transaction (≥ 0)
transactions	governmentTax	DECIMAL(10,2)		Government tax applied (≥ 0)
transactions	senderBalanceAfter	DECIMAL(12,2)		Sender balance after transaction
transactions	receiverBalanceAfter	DECIMAL(12,2)		Receiver balance after transaction
transactions	referenceText	VARCHAR(255)		Free-text reference or note for the transaction
transaction_categories	transCategoryId	INT	PK	Unique identifier for the category
transaction_categories	transCategoryName	VARCHAR(50)		Name of the category e.g. Transfer, Payment
transaction_categories	description	VARCHAR(255)		Description of what the category covers
transaction_map	transactionId	INT	PK, FK -> transactions.transactionId	Links to a transaction
transaction_map	transCategoryId	INT	PK, FK -> transaction_ca	Links to a category

			tegories.trans CategoryId	
system_logs	sysLogId	INT	PK	Unique identifier for the log entry
system_logs	transactionId	INT	FK -> transactions.tr ansactionId (nullable)	The related transaction, if any
system_logs	logLevel	VARCHAR(20)		Severity level: INFO, WARN, ERROR
system_logs	message	VARCHAR(255)		Log message detail
system_logs	logTimestamp	DATETIME		Timestamp when the log was recorded

Sample Query demonstrating database functionalities:

1. CREATE: Investigate a new transaction:

```

86 CREATE INDEX idx_transactions_status ON transactions(status);
87 CREATE INDEX idx_systemlogs_loglevel ON system_logs(logLevel);
88
89 INSERT INTO transactions (senderId, receiverId, transactionTimestamp, status, currentAmount, transac
90 VALUES (2, 3, '2026-06-05 12:00:00', 'SUCCESS', 1000.00, 20.00, 2.00, 33780.00, 3500.00, 'Transfer t

```

Result

```

CREATE INDEX idx_transactions_status ON transactions(status)
AffectedRows: 0

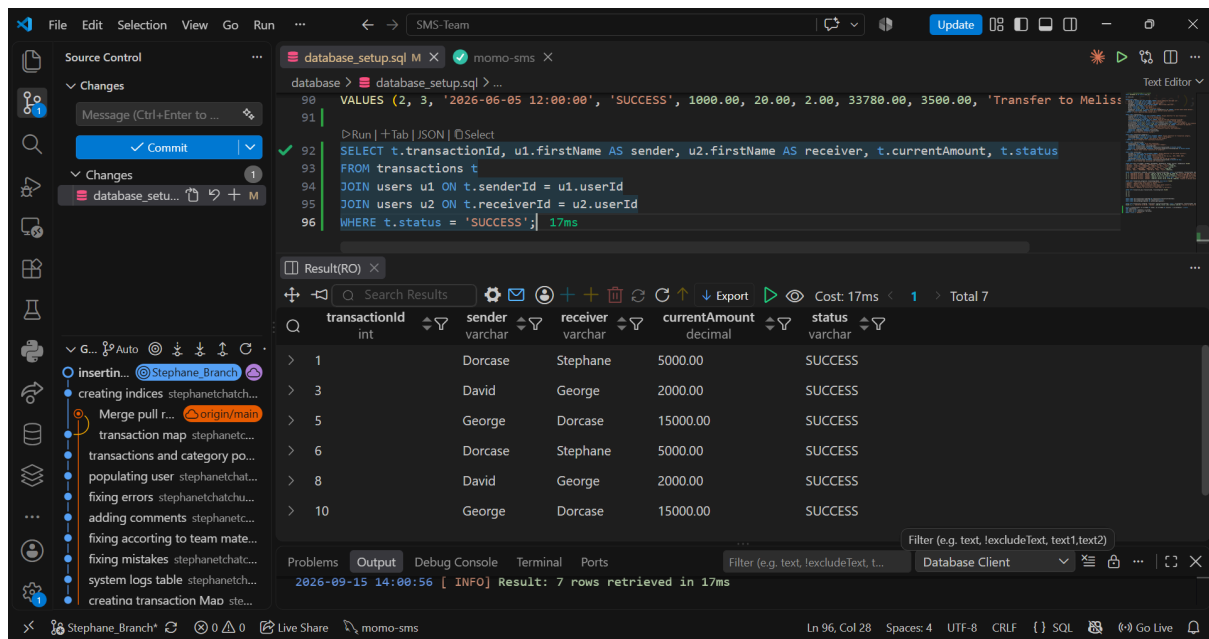
```

2026-09-15 13:57:21 [INFO] Executing: INSERT INTO transactions (senderId, receiverId, transactionTimestamp, status, currentAmount, transactionFee, governmentTax, senderBalanceAfter, receiverBalanceAfter, referenceText) VALUES (2, 3, '2026-06-05 12:00:00', 'SUCCESS', 1000.00, 20.00, 2.00, 33780.00, 3500.00, 'Transfer to Melissa Elise')

2026-09-15 13:57:21 [INFO] Result: 1 rows affected in 22ms

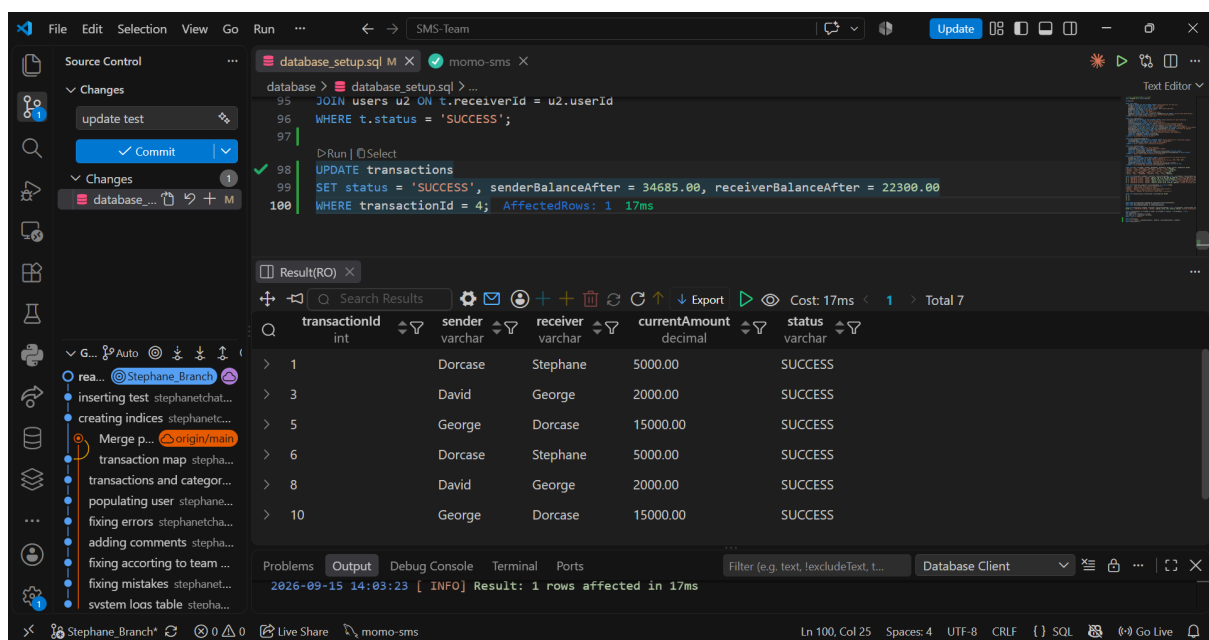
INSERT statement adding a new transaction record, 1 row affected

2. READ: Retrieving all successful transactions with sender and receiver names via JOIN across Transaction and User



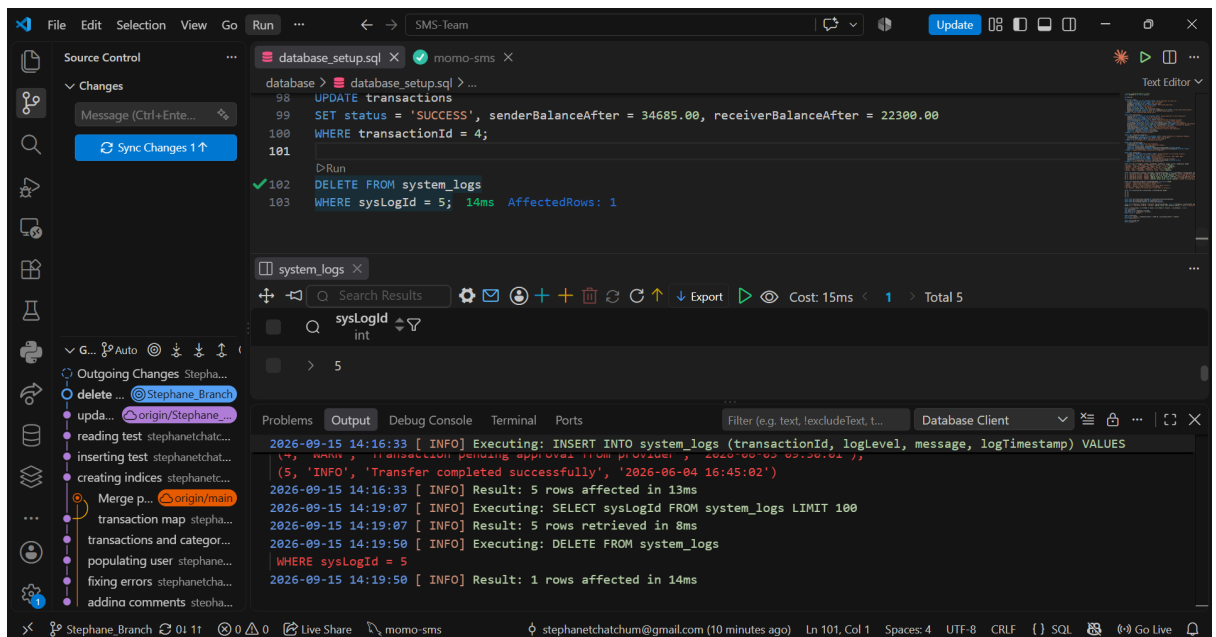
READ statement printing all successful transactions across the transaction and user table

3. UPDATE: Changing the status, senderBalanceAfter and receiverBalanceAfter for transactionId = 4



UPDATE statement to modify some entries and specify which one using the WHERE clause

4. DELETE: Removing one of the entries from system_logs



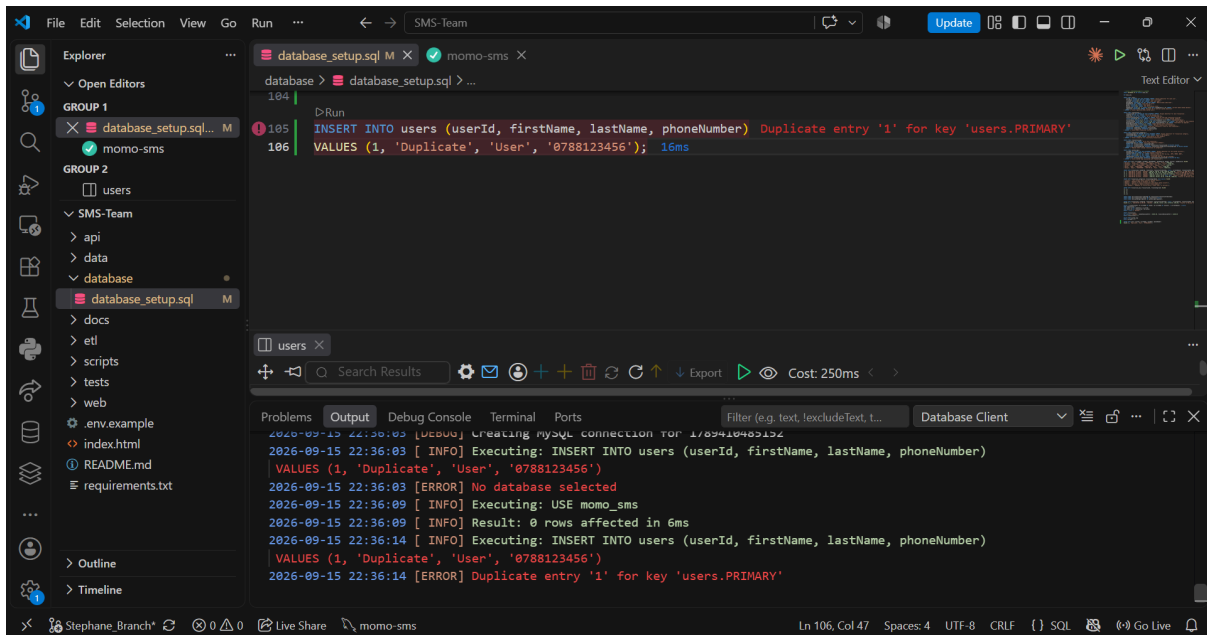
The DELETE statement is used to remove an entry, followed by WHERE to specify, using the primary key, which entry to delete.

Unique Rules Added to Enhance Security and Accuracy of the Database

The database includes several constraints and rules designed to maintain accurate, consistent, and reliable data. These rules are implemented directly in the database structure.

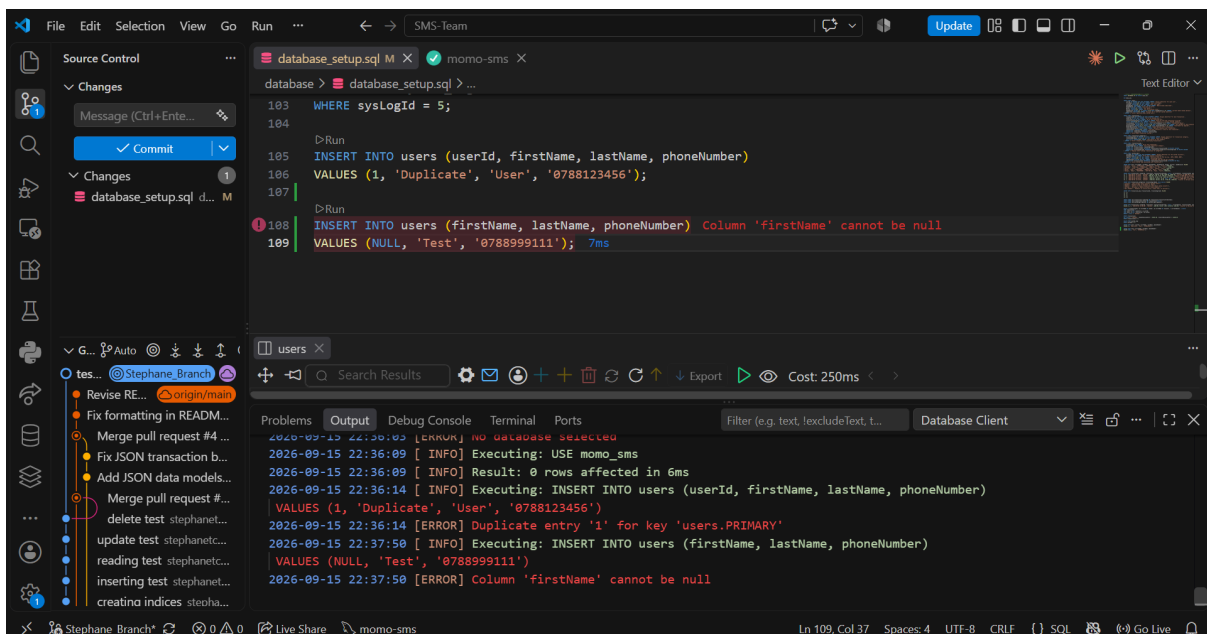
1. Primary Key Constraints

A primary key gives each record a unique identity. This makes it easier to identify each record and prevents duplicate identification. In the database, `userId` and `transactionId` are examples of primary keys.



2. NOT NULL Constraints

NOT NULL is essential where it makes sure that the fields are not be left empty. This keeps the required data accurate.



3. UNIQUE Constraint

The **UNIQUE** rule makes sure that a value cannot be repeated where it must be different. Like for example, each user's phone number must be different.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Source Control' tab with a list of changes. The main pane shows the 'database_setup.sql' script with the following SQL code:

```
database > database_setup.sql > ...
106 INSERT INTO users (userId, firstName, lastName, phoneNumber)
107 VALUES (1, 'Duplicate', 'User', '0788123456');
108
109 -- Run
110 INSERT INTO users (firstName, lastName, phoneNumber)
111 VALUES (NULL, 'Test', '0788999111');
112
113 -- Run
114 INSERT INTO users (firstName, lastName, phoneNumber)
115 VALUES ('Fake', 'User', '078811222'); 10ms
```

The 'Problems' pane at the bottom shows the following error:

```
2026-09-15 22:37:50 [ERROR] Duplicate entry '078811222' for key 'users.phoneNumber'
```

4. CHECK Constraints

CHECK rules make sure that values follow certain conditions. In this database, they help prevent invalid negative financial values such as balances, transaction amounts, fees, and taxes

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Source Control' tab with a list of changes. The main pane shows the 'database_setup.sql' script with the following SQL code:

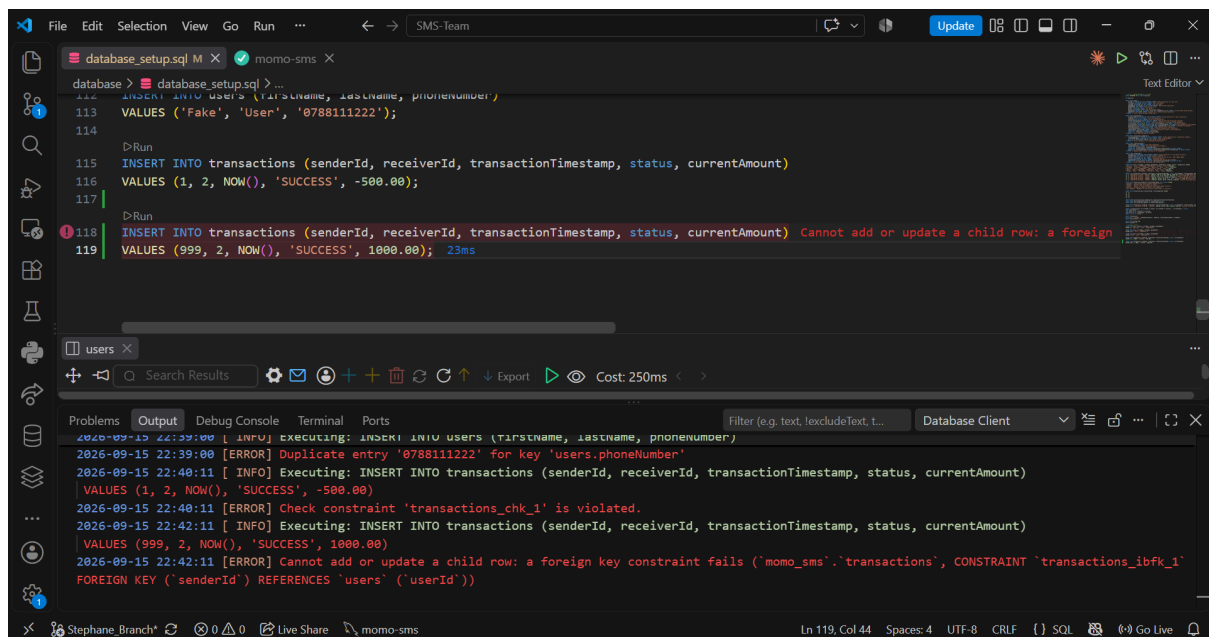
```
database > database_setup.sql > ...
106 INSERT INTO users (userId, firstName, lastName, phoneNumber)
107 VALUES (1, 'Duplicate', 'User', '0788123456');
108
109 -- Run
110 INSERT INTO users (firstName, lastName, phoneNumber)
111 VALUES (NULL, 'Test', '0788999111');
112
113 -- Run
114 INSERT INTO users (firstName, lastName, phoneNumber)
115 VALUES ('Fake', 'User', '078811222');
116
117 -- Run
118 INSERT INTO transactions (senderId, receiverId, transactionTimestamp, status, currentAmount)
119 VALUES (1, 2, NOW(), 'SUCCESS', -500.00); 21ms
```

The 'Problems' pane at the bottom shows the following error:

```
2026-09-15 22:40:11 [ERROR] Check constraint 'transactions_chk_1' is violated.
```

5. Foreign Key Constraints

Foreign keys connect related tables. They make sure that a record refers to a valid record in another table



The screenshot shows an IDE with a SQL editor and a console. The SQL editor contains the following code:

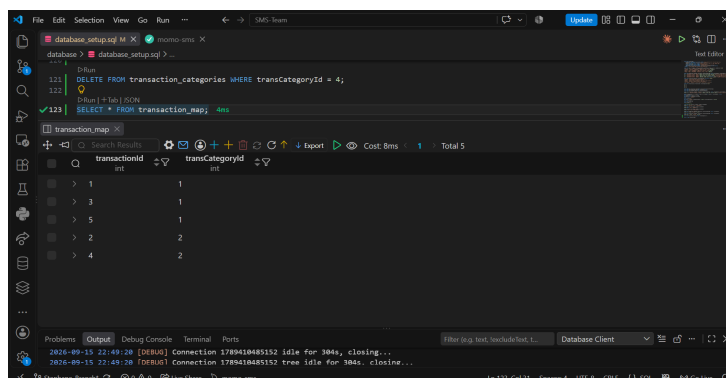
```
database > database_setup.sql >...
113 INSERT INTO users (firstname, lastname, phonenumber)
114 VALUES ('Fake', 'User', '0788111222');
115 >Run
116 INSERT INTO transactions (senderId, receiverId, transactionTimestamp, status, currentAmount)
117 VALUES (1, 2, NOW(), 'SUCCESS', -500.00);
118 >Run
119 INSERT INTO transactions (senderId, receiverId, transactionTimestamp, status, currentAmount)
120 VALUES (999, 2, NOW(), 'SUCCESS', 1000.00); 23ms
```

The console shows the following error messages:

```
2026-09-15 22:39:00 [ERROR] Duplicate entry '0788111222' for key 'users.phoneNumber'
2026-09-15 22:40:11 [INFO] Executing: INSERT INTO transactions (senderId, receiverId, transactionTimestamp, status, currentAmount)
VALUES (1, 2, NOW(), 'SUCCESS', -500.00)
2026-09-15 22:40:11 [ERROR] Check constraint 'transactions_chk_1' is violated.
2026-09-15 22:42:11 [INFO] Executing: INSERT INTO transactions (senderId, receiverId, transactionTimestamp, status, currentAmount)
VALUES (999, 2, NOW(), 'SUCCESS', 1000.00)
2026-09-15 22:42:11 [ERROR] Cannot add or update a child row: a foreign key constraint fails ('momo_sms`.`transactions`, CONSTRAINT `transactions_ibfk_1'
FOREIGN KEY (`senderId`) REFERENCES `users` (`userId`))
```

6. ON DELETE CASCADE

ON DELETE CASCADE automatically removes related records when the main record they depend on is deleted. This helps prevent unnecessary or orphaned records.



The screenshot shows an IDE with a SQL editor and a console. The SQL editor contains the following code:

```
database > database_setup.sql >...
121 DELETE FROM transaction_categories WHERE transCategoryId = 4;
122 >Run
123 SELECT * FROM transaction_map; 4ms
```

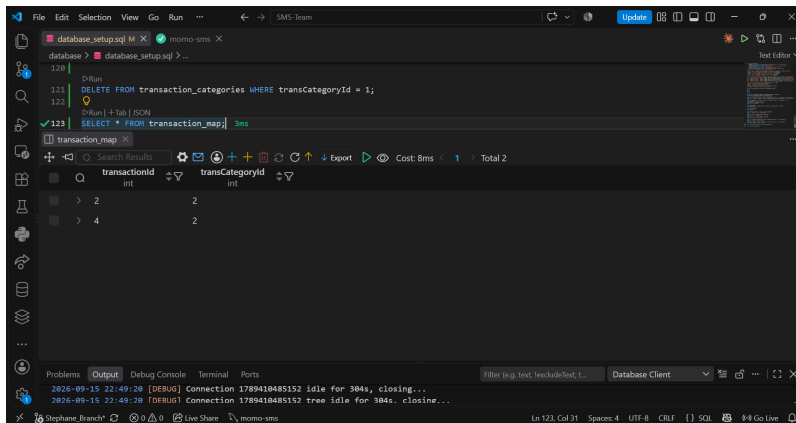
The console shows the following error messages:

```
2026-09-15 22:49:20 [DEBUG] Connection 1789010401502 idle for 306s, closing...
2026-09-15 22:49:20 [DEBUG] Connection 1789010401512 idle for 346s, closing...
```

The table view shows the following data:

transCategoryId	transCategoryId
1	1
3	1
5	1
2	2
4	2

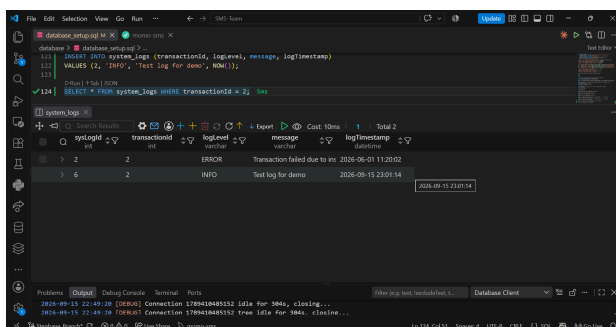
Before deleting



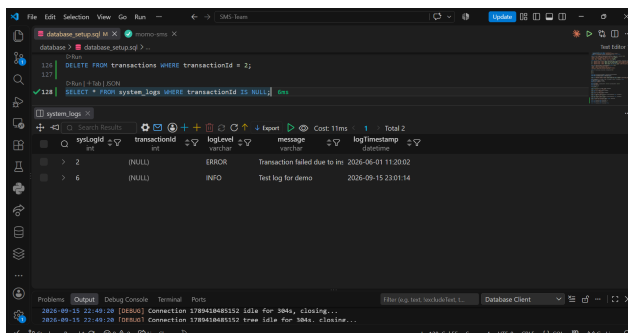
After deleting

7. ON DELETE SET NULL

ON DELETE SET NULL changes the related foreign-key value to NULL when the referenced record is deleted. This allows the related record to remain in the database.



Before deleting



After deleting

AI Usage Log

Dorcase Lesly Nana Tounda (ERD Design & Documentation)

Entity identification, attribute selection, relationship design (including the sender/receiver modeling approach and the many-to-many resolution via the Transaction_Map junction table), and the written design rationale were independently researched and developed.

Research was conducted directly from MTN Rwanda/Uganda official documentation and an IMF working paper on mobile money (see References section). AI (Claude) was used to:

- Validate and stress-test design decisions already made (e.g. confirming whether a composite primary key or separate FK columns was appropriate for the Transactions table, checking whether the M:N relationship was correctly identified and resolved)
- Check terminology and explain relational database concepts (e.g. the distinction between a foreign key's column name and the primary key values it references) for personal understanding
- Proofread and format the final design rationale paragraph for clarity and word count compliance. AI-suggested schema drafts were reviewed but not used in the final submitted ERD; the submitted diagram and rationale reflect independent design work, checked against AI feedback for errors.

Tchatchum Stephane (SQL Implementation)

Used AI (Claude) to check SQL syntax of CREATE TABLE, INSERT, and CREATE INDEX statements written independently, and to flag errors (e.g. inconsistent table name casing, incorrect data type for phoneNumber, missing constraints). Used AI to verify logical/data consistency in sample INSERT rows (e.g. balance arithmetic across transactions). Used AI to troubleshoot a MySQL connection issue during CRUD testing where a DELETE query returned 0 rows affected. Used AI to generate the data dictionary table formatting from the finalized schema, for inclusion in the Database Design Document. All table design, relationships, and business logic were defined by the team (see Dorcase's ERD and design rationale); AI was not used to generate schema design or transaction logic.

Migisha Olivier (JSON Data Modeling)

Used AI (Claude) to check the syntax and validity of the JSON structure written for the MoMo SMS database entities, verify that JSON field names and values were consistent with the finalized SQL database structure, and check transaction data for consistency (related user information, transaction categories, balance values). AI was also used to identify and correct inconsistent transaction balance values in the JSON sample data, and to verify the completed JSON file using a JSON validation command before pushing to GitHub. All JSON data modeling decisions, entity relationships, field selection, and business logic were based on the team's finalized database design; AI was used for syntax checking and consistency checking only.

References

MTN Rwanda. (2024). *MoMoCard terms and conditions*.

<https://www.mtn.co.rw/wp-content/uploads/2024/10/MOMOCARD-TCs.pdf>

MTN Rwanda. (2023). *MoMo service charter*.

<https://www.mtn.co.rw/wp-content/uploads/2023/08/MoMo-Service-Charter.pdf>

MTN Uganda. (2024). *MoMo agent agreement booklet*.

<https://www.mtn.co.ug/wp-content/uploads/sites/7/2024/05/MoMo-Agent-Agreement-Booklet.pdf>

MTN Uganda. (2019). *MTN mobile money consumer terms and conditions*.

<https://www.mtn.co.ug/wp-content/uploads/2019/10/MTN-MOBILE-MONEY-Consumer-Terms-and-Conditions-0519.pdf>

Shirono, K., Das, B., Fan, Y., Chhabra, E., & Carcel-Villanova, H. (2021). *Is mobile money part of money? Understanding the trends and measurement* (IMF Working Paper WP/21/177). International Monetary Fund.

<https://www.elibrary.imf.org/view/journals/001/2021/177/article-A001-en.xml>